



MORINGA

Discover · Grow · Transform



Classification ML & Hyperparameter tuning



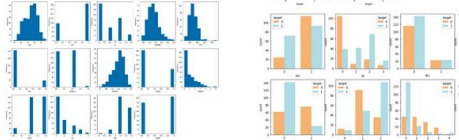
Session Expectations

Online edition	The usual
→ Switch on your video when speaking (if possible) → Participate in the discussions → Mute yourself when you are not speaking	→ Be present → Be honest → Be open → Be curious

Classification Models

Exploratory Data Analysis (EDA)

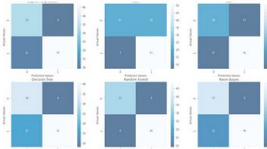
- 1) Histogram: `df.plot(kind = 'hist')`
- 2) Box Plot: `sns.boxplot()`
- 3) Grouped Bar Chart: `sns.countplot()`



Model Evaluation

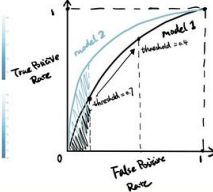
Confusion Matrix

`confusion_matrix(y_test, y_pred)`

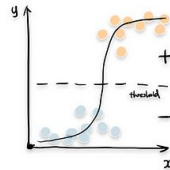


ROC & AUC

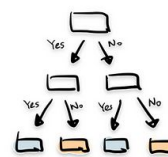
`metrics.auc(fpr, tpr)`



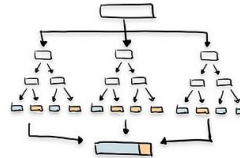
Logistic Regression



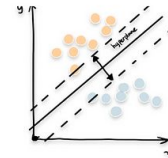
Decision Tree



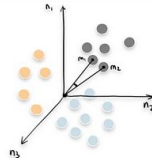
Random Forest



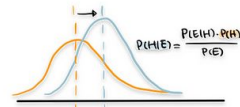
Support Vector Machine



K Nearest Neighbour



Naive Bayes



The model tries to predict the correct label of a given input data

○ Logistic Regression



- uses the logistic/sigmoid function to map predictions to probabilities.
- output is between 0 and 1, which is suitable for binary classification.

Parameters

- penalty: L1 or L2 regularization or no regularization.
- C: The inverse of regularization strength..
- solver:
- max_iter:
- class_weight:

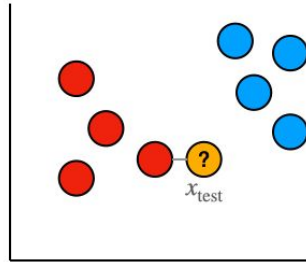
K-Nearest Neighbour (KNN)

KNN predicts the class of a new data point by a majority vote among its K nearest neighbors, where K is a hyperparameter chosen by the user.

performance can be sensitive to the choice of the number of neighbors (K) and the distance metric.

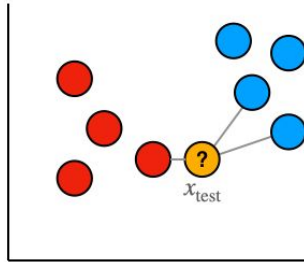
Parameters:

- `n_neighbors`,
- `weights`: 'uniform' or 'distance'
- `algorithm`: euclidian, manhattan,
- `metric`,
- `leaf_size`,
- `p`



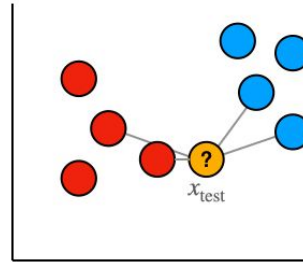
$k = 1$

Nearest point is **red**, so x_{test} classified as **red**



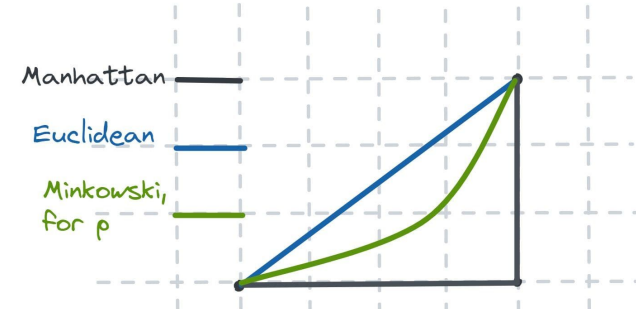
$k = 3$

Nearest points are {**red**, **blue**, **blue**} so x_{test} classified as **blue**



$k = 4$

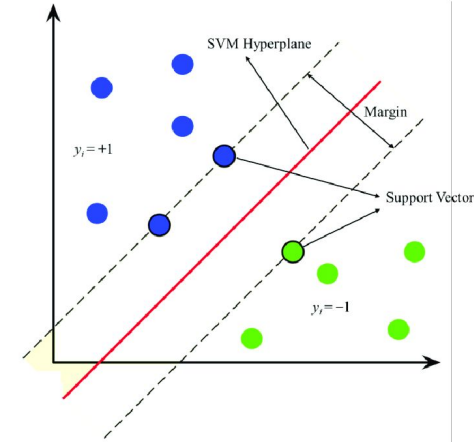
Nearest points are {**red**, **red**, **blue**, **blue**} so classification of x_{test} is not properly defined



Minkowski when $p = 1$ --> Manhattan
 Minkowski when $p = 2$ --> Euclidean

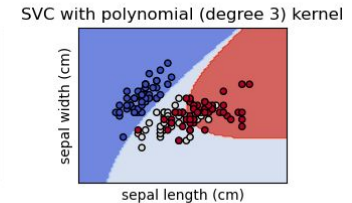
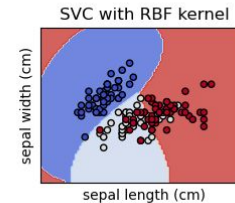
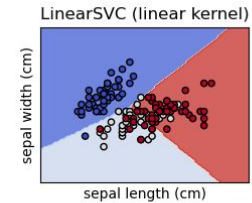
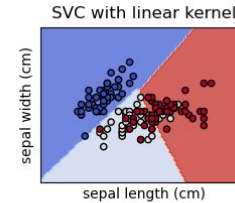
Support Vector Machine

- Works by finding the optimal hyperplane that separates data points of different classes with the maximum margin.
- objective of SVM is to maximize the margin between the hyperplane and the closest data points (support vectors). better generalization and robustness of the model.
- highly dependent on the choice of parameters, particularly C, kernel type, and gamma.



Parameters:

- C: The penalty parameter. It controls the trade-off between maximizing the margin and minimizing the classification error.
- kernel: 'linear', 'poly', 'rbf', and 'sigmoid'
- gamma
- degree
- class_weight:
- decision_function_shape:

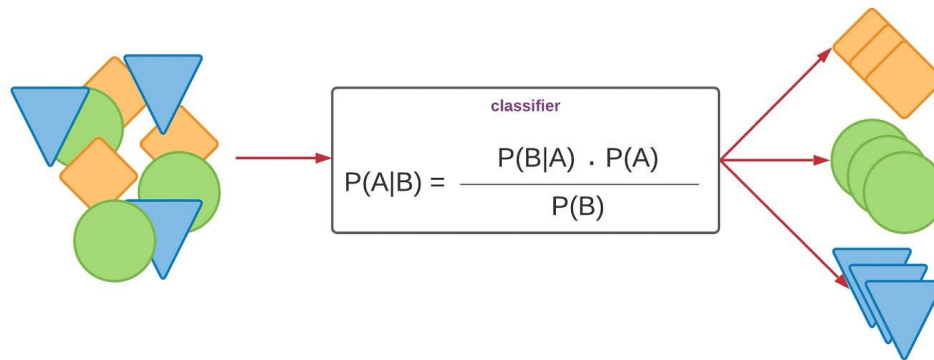


Naive Bayes

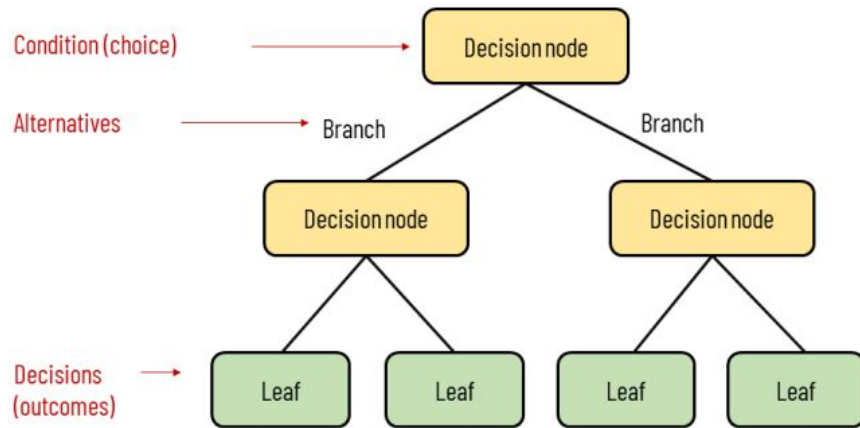
- Based on Bayes' theorem with the "naive" assumption of independence between features.
- Calculates the probability of each class given the input features and select the class with the highest probability as the prediction
 - **Gaussian Naive Bayes**: Assumes that continuous features follow a Gaussian (normal) distribution.
 - **Multinomial Naive Bayes**: Suitable for discrete features, often used in text classification with term frequencies as features.
 - **Bernoulli Naive Bayes**: Assumes binary features (e.g., presence or absence of a term), often used in binary classification tasks.

Parameters: They do not have as many hyperparameters as some other algorithms due to their simplicity and the nature of their probabilistic model.

- alpha, fit_prior, class_prior:



○ Decision Tree



- a step-by-step graphical approach that works on the probable outcomes.

Parameters

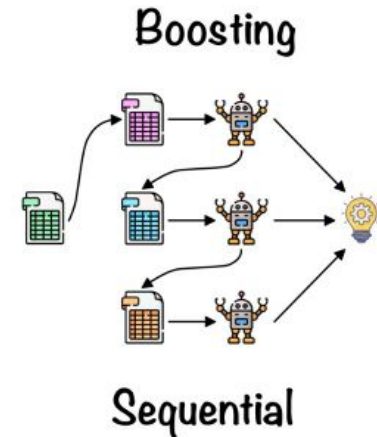
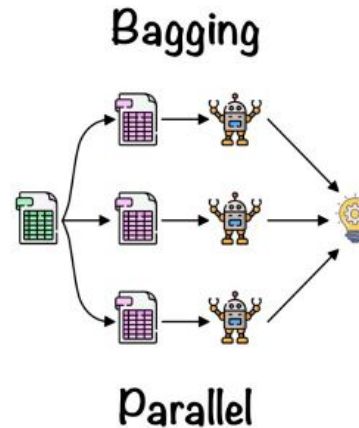
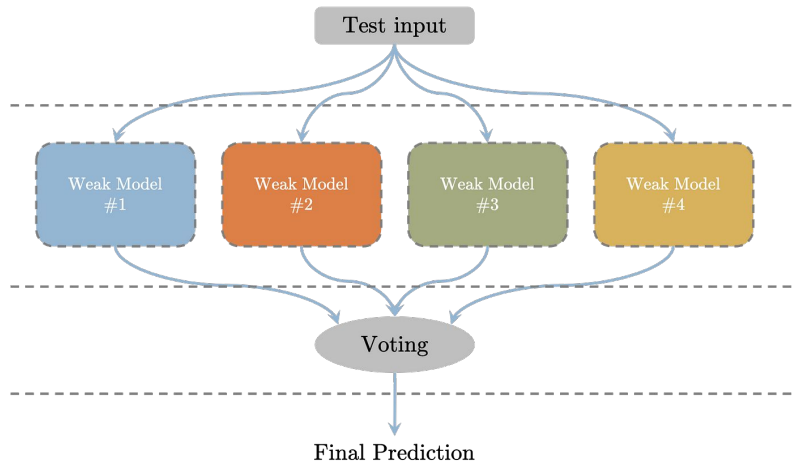
- criterion: gini/ entropy
- max_depth
- min_samples_split
- min_samples_leaf
- max_features:



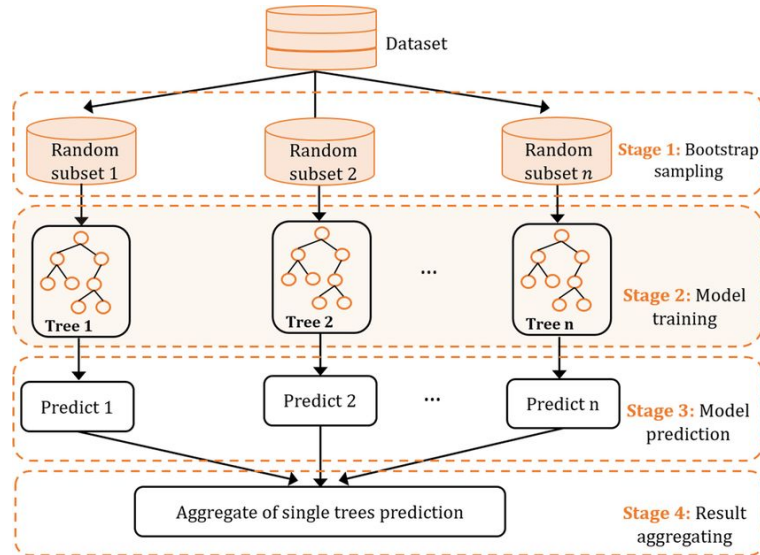
Ensemble methods

Techniques that create multiple models and then combine them to produce improved results.

- Stacking: combines the predictions of multiple diverse models using another meta-model.
- Voting: Voting ensembles combine predictions from multiple heterogeneous models



Bagging (bootstrap aggregating)



Random Forest creates multiple decision trees, typically known as the forest.

Each decision tree is built using a subset of the training data sampled with replacement (bootstrap samples).

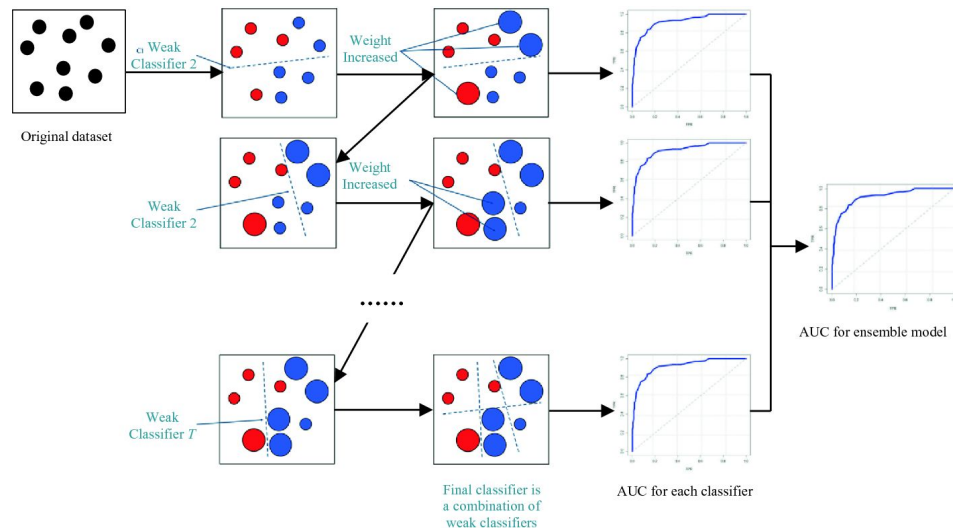
Parameters

- `n_estimators`: n of decision trees in the forest.
- `criterion`
- `max_depth`
- `min_samples_split`
- `min_samples_leaf`
- `max_features`
- `bootstrap`: If True, each tree is built on a bootstrap sample of the training data. If False, the whole dataset is used to build each tree.
- `random_state`

Boosting

1. AdaBoost (Adaptive Boosting)

- Sequentially trains a series of weak learners on repeatedly modified versions of the data.
- Each weak learner is trained to focus on the instances that were misclassified by the previous ones, assigning higher weights to these instances.
- The final prediction is a weighted sum of the predictions of all weak learners.
- Effective on models with high bias and low variance.



Boosting Parameters:

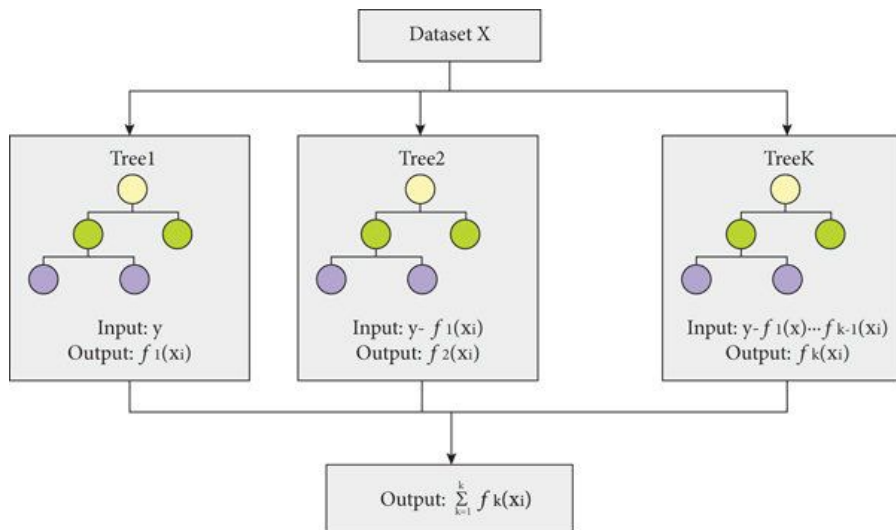
- `n_estimators`: The number of boosting stages (trees) to be used.
- `learning_rate`
- `max_depth`: The maximum depth of each tree.
- `subsample`: The fraction of samples used to fit each tree.

2. Gradient Boosting

- builds a series of decision trees sequentially, with each tree fitting the residuals
- directly minimizes a loss function by adding new models that correct the errors of the previous ones.
- **Con** -can be computationally expensive and prone to overfitting

3. XGBoost

- optimized implementation of Gradient Boosting
- advanced regularization techniques such as shrinkage (learning rate) and column subsampling (feature importance)
- highly scalable and efficient; suitable for large datasets and real-world applications.

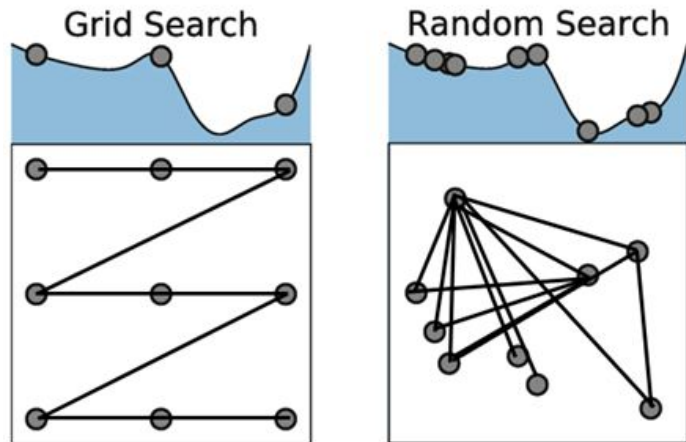


Hyperparameter tuning

The process of finding the optimal values for the hyperparameters of a machine learning model.

- A. **Grid Search**: Exhaustively search through a manually specified subset of the hyperparameter space.
- B. **Random Search**: Randomly sample from the hyperparameter space. It's often more efficient than grid search, especially in high-dimensional spaces.
- C. **Bayesian Optimization**: Use probabilistic models to select the next set of hyperparameters to evaluate based on past evaluations' performance.
- D. **Genetic Algorithms**: Mimic the process of natural selection to iteratively improve a set of candidate solutions

Perform cross-validation to evaluate the model's performance for each set of hyperparameters.



End!

